MetraAI — Final Technical Stack

Legal Metrology AI Compliance System — SIH Problem Statement 26034

Final stack optimized for HTML/CSS/JavaScript frontend and Python backend, aligned with the complete MetraAI inspection, AI evidence extraction, deterministic compliance, human review, reporting and audit workflow.

1. Final Stack at a Glance

Layer	Technology	Purpose
Frontend	HTML5 + CSS3 + JavaScript	Web application and officer interface
UI	Bootstrap 5	Responsive professional UI
Charts	Chart.js	Dashboard analytics and visualizations
Backend	Python + FastAPI	REST API and application orchestration
Validation	Pydantic	Request/response and structured-data validation
ORM / Migrations	SQLAlchemy + Alembic	Database access and schema migrations
OCR	PaddleOCR	Multilingual OCR, text and bounding boxes
Computer Vision	OpenCV	Pre-processing, quality, readability and visual analysis
Object Detection	YOLO	Package/label/region detection
NLP	spaCy + Transformers	Entity/declaration extraction
Structured Extraction	Regex + deterministic parsers + Pydantic	Reliable field extraction
LLM	Gemini / OpenAI / OpenRouter	Ambiguity resolution and explanations
Embeddings	Sentence Transformers	Legal-document embeddings
Legal RAG	pgvector + retrieval layer	Relevant rule/source retrieval
Rule Engine	Python deterministic rule engine	Actual compliance evaluation
Database	PostgreSQL + pgvector	Application, rule and vector data
Background Jobs	Celery + Redis	Async OCR/AI processing
Storage	MinIO / S3	Images, evidence and reports
Barcode / QR	pyzbar / compatible decoder	Product identifier decoding
Reports	ReportLab	Compliance PDF generation
Authentication	JWT + RBAC	Secure role-based access
Audit	PostgreSQL audit tables	Inspection/review history
Testing	Pytest + Playwright	Backend/AI and end-to-end testing
Deployment	Docker + Docker Compose	Reproducible deployment
CI/CD	GitHub Actions	Automated tests and builds
Monitoring	Sentry + structured logging	Error tracking and observability
Version Control	Git + GitHub	Team collaboration

2. Architecture

```
FRONTEND
HTML5 + CSS3 + JavaScript + Bootstrap + Chart.js
↓ REST API / JSON
BACKEND
Python + FastAPI + Pydantic + SQLAlchemy
↓
PostgreSQL + pgvector    |    Redis + Celery    |    MinIO/S3
```

```
↓
AI WORKERS
OpenCV + PaddleOCR + YOLO + spaCy/Transformers
↓
Structured Product Record + Evidence
↓
Barcode/QR Cross-check + Product Classification
↓
Rule Applicability Engine
↓
Versioned Deterministic Legal Rule Engine
↓
NO ISSUE / REVIEW REQUIRED / POTENTIAL NON-COMPLIANCE
↓
Human Officer Review
↓
Report + Case + Escalation + Dashboard + Audit Trail
```

3. Frontend

HTML5 + CSS3 + JavaScript ES6+ are the primary frontend technologies. Bootstrap 5 can provide responsive components and Chart.js can provide dashboard charts. React is intentionally not required because the team is already strong in vanilla web technologies.

Recommended pages

`/login` `/dashboard` `/inspection/new` `/inspection/:id` `/products` `/violations` `/cases` `/reports` `/rules` `/audit`

4. Backend

Python + FastAPI should be the central backend because the AI/CV pipeline is Python-based. FastAPI exposes REST endpoints while Pydantic handles validation and SQLAlchemy handles database access.

Core API areas: authentication, inspections, products, images, OCR, declarations, rules, compliance, violations, evidence, cases, reports, audit and analytics.

5. AI Evidence Extraction Pipeline

```
Input Image → OpenCV Quality Check → Pre-processing → Text Detection → PaddleOCR → Information Extraction → Confidence Scoring → Structured Product Record.
```

Image Processing — OpenCV

Use OpenCV for denoising, perspective correction, contrast enhancement, blur/focus checks, glare detection, image resolution checks and readability analysis.

OCR — PaddleOCR

PaddleOCR provides multilingual text extraction, confidence scores and bounding boxes. Bounding boxes must be preserved because they support evidence highlighting, placement checks and font-size estimation.

```
{"text":"MRP ■120","confidence":0.97,"bbox":[120,300,420,350]}
```

Computer Vision — YOLO

YOLO is used for package, label and declaration-region detection. It should provide visual localization rather than making the legal compliance decision.

6. Information Extraction

Use a layered extraction strategy: (1) Regex/deterministic parsers for MRP, phone numbers, email, dates, quantity and units; (2) spaCy/Transformers for entities such as manufacturer, packer, importer and country of origin; (3) an LLM only when deterministic extraction is ambiguous; (4) Pydantic validation before data enters the compliance engine.

7. Structured Product Record

Store product name, brand, manufacturer, packer, importer, country of origin, net quantity, MRP, unit sale price where applicable, batch number, manufacturing/packing/expiry information, consumer-care details, ingredients and other declarations. Every extracted field should also store confidence, source, bounding box and manual-verification status.

8. Barcode / QR and Product Cross-check

Decode product identifiers using a barcode/QR library and compare trusted-source product information against OCR-derived values. Store the source, retrieval timestamp and confidence; do not blindly trust an external database.

9. Legal Rule Applicability Engine

Determine which provisions apply using product category, pack type, sales channel, inspection date, conditions and exceptions. This prevents irrelevant legal provisions from being evaluated.

10. Versioned Deterministic Rule Engine

This is the authority for automated compliance evaluation. Each rule should maintain rule ID, version, requirement, applicability, effective/expiry dates, exceptions, source and amendment reference.

Each evaluation returns exactly one operational state: NO_ISSUE, REVIEW_REQUIRED, or POTENTIAL_NON_COMPLIANCE, together with evidence, confidence and rule version.

11. Legal RAG

Official legal documents → text extraction → chunking → Sentence Transformers embeddings → PostgreSQL/pgvector → retrieval → LLM explanation. RAG should explain and retrieve legal sources; it should not replace the deterministic rule engine.

12. Human-in-the-Loop

AI assists the officer; the final legal decision remains human-validated. Officer actions should include confirm no issue, mark review required, confirm potential non-compliance, correct extracted data, reject an AI suggestion and add remarks. Every decision must enter the audit trail.

13. Database and Storage

PostgreSQL stores users, inspections, products, OCR results, declarations, rules, rule versions, compliance checks, violations, evidence, cases, reports, legal documents and audit logs. **pgvector** stores legal-document embeddings. Large images and reports should live in **MinIO/S3**, while PostgreSQL stores their metadata and object references.

14. Background Processing

Use Celery + Redis so OCR and AI processing does not block the API. Upload → FastAPI → job → Redis → Celery worker → OpenCV/OCR/NLP/compliance → PostgreSQL → frontend.

15. Font Size and Readability

Font-size checking should use OCR bounding boxes, image measurements and calibration/reference information. A normal photograph does not directly provide physical millimetres, so the system should support PASS / FAIL / REVIEW_REQUIRED rather than claiming exact measurements without adequate calibration.

Readability checks: contrast, blur, sharpness, glare, background clutter and text visibility.

16. E-Commerce Inspection

Support listing screenshots/uploads first. Where URL extraction is implemented, comply with the target site's terms, robots policies, authentication requirements and applicable law. Extract title, description, highlights and listing images, then pass the result through the same structured-record and compliance pipeline.

17. Reports, Cases and Escalation

ReportLab generates PDF compliance reports containing inspection information, product details, extracted declarations, rule results, evidence, rule versions, officer decision, remarks and timestamps. Confirmed non-compliance creates a Case ID, evidence pack, remarks and escalation status.

18. Security and Audit

Use JWT authentication, RBAC, password hashing, Pydantic validation, file-type/size validation, rate limiting, CORS and HTTPS in production. Roles can include ADMIN, OFFICER and REVIEWER. Audit who changed what, when, previous/new values, decisions, reasons, AI results, officer results and rule versions.

19. Testing and Deployment

Backend/AI: Pytest. Frontend: browser-level and component testing as needed. End-to-end: Playwright. Use Docker and Docker Compose for reproducible local deployment. GitHub Actions should run linting, tests, integration checks and Docker builds.

20. Final Architecture Principle

AI is not the legal decision-maker.

AI extracts, detects and assists → structured evidence → rule applicability → deterministic legal rule engine → AI/RAG explanation → human officer review → final decision → report/case/audit trail.

Recommended SIH MVP: HTML/CSS/JavaScript + FastAPI + PostgreSQL + OpenCV + PaddleOCR + deterministic rule engine + evidence visualization + human review + PDF reporting. Add YOLO, RAG and LLM capabilities after the core

compliance pipeline is stable.